Welcome Back

Recap

- ⟶ LLM foundation
- ⟶ API keys
- ⟶ Used the Chat models
- ⟶ Used hugging face and local model
- ⟶ Used messages
- ⟶ Prompts
- ⟶ Structured output

Now you know me guys
we have to complete everything
about RAG in this video but while
teaching Generative AI I got to
know the best way of teaching
this is project based learning

So Lets start with a great project

CourseMate AI

CourseMate AI is an AI-powered study assistant designed to help students interact with their learning materials more efficiently. Modern students rely on multiple sources of study material such as lecture notes, textbooks, PDFs, and research papers. These documents are often long and difficult to navigate, making it time-consuming to find specific information.

CourseMate AI

CourseMate AI aims to solve this problem by allowing students to chat with their study materials. Instead of manually searching through pages of content, students can simply ask questions and receive accurate answers extracted directly from their documents.

CourseMate AI

By leveraging Retrieval-Augmented Generation (RAG), CourseMate AI combines document retrieval with large language models to provide context-aware explanations, summaries, and answers from the student's own study resources.

Development plan

Now to develop this kind of system we are going to follow some steps and the good thing is we are going to apply each and every step and create our project as well.

toh padhai bhi ho jaaye gi and project bhi ban jaaye ga.

Development plan

Step 1 - User Uploads Study Material

User Uploads Study Material
Students upload learning resources such as:

- PDFs
- lecture notes
- textbooks
- research papers

Development plan

Step 2 - Document Loading

The system loads documents using document loaders.
Goal:
Convert raw files into document objects that can be processed.

you may clean the document as well

Development plan

Step 3 - Text Splitting (Chunking)

Documents are usually too large for LLM context windows.
So we split them into smaller chunks.

Chunking improves retrieval accuracy.

Development plan

Step 4 - Embedding Generation

Each chunk is converted into a vector embedding.
Embedding models transform text into numerical vectors.

"Gradient Descent Optimization"
↓
[0.23, -0.81, 0.44, ...]

These vectors represent semantic meaning.

Development plan

Step 5 - Vector Database Storage

All embeddings are stored inside a vector database.
The vector database stores:

- embeddings
- original text chunks
- metadata

Development plan

Step 6 - User Asks a Question

Now the student interacts with the system.

Development plan

Step 7 - Query Embedding

The question is also converted into an embedding.

Development plan

Step 8 - Similarity Search

The vector database performs semantic similarity search.
Goal:
Find chunks that are most relevant to the question.

Development plan

Step 9 - Retriever Component

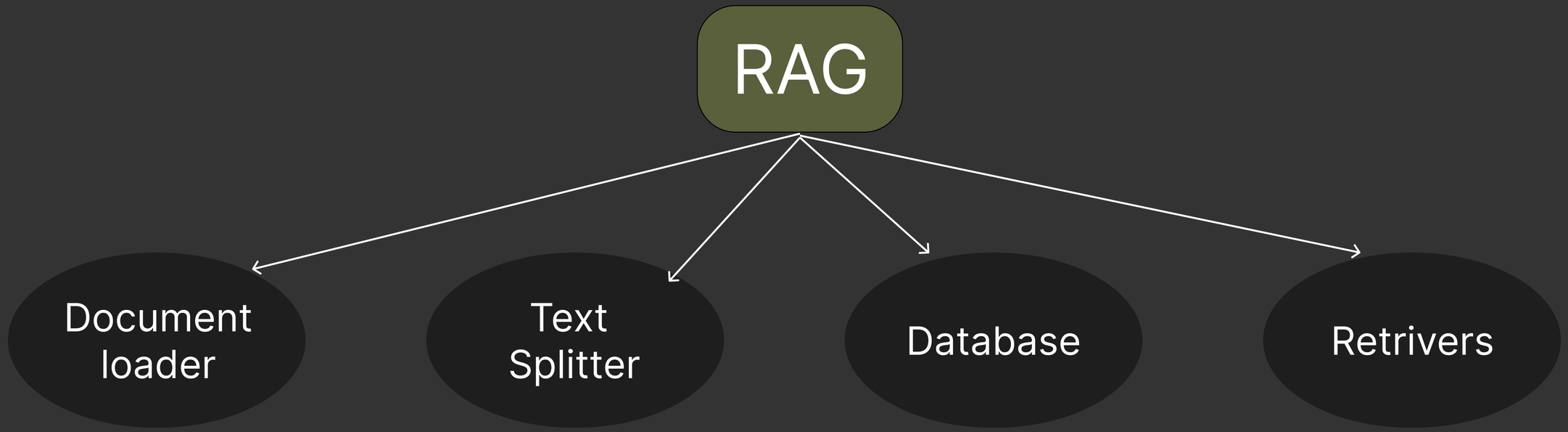
The retriever selects the top-k relevant chunks.
These chunks form the context.

Development plan

Step 10 - LLM answers

Based on the context the LLM answers.

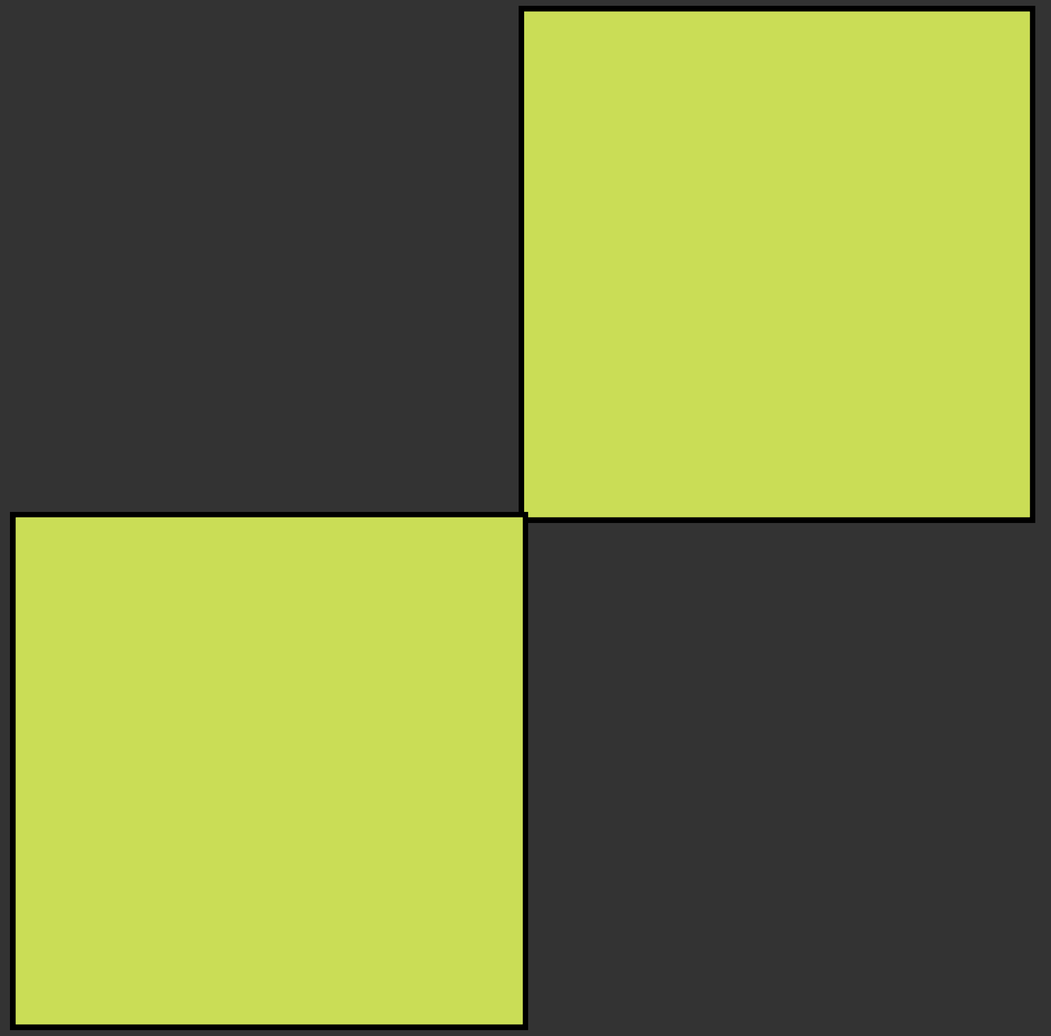
RAG



Retrieval Augmented Generation

Lets Go...

Ok so now lets create this and also I will teach each and every thing while making the project all the best.



Text Splitter

1. Most of the embedding models and language model have a context window and they cannot take infinite tokens at the same time so for that we have to use the text splitting.

Text Splitter

2. Better Retrieval in RAG

In Retrieval-Augmented Generation (RAG), we search through vector embeddings.

If the document is too large:

- embeddings become less precise

Smaller chunks allow the system to retrieve only the most relevant piece of information instead of the whole document.

Text Splitter

3. More Accurate Embeddings

Embeddings work best when the text represents one clear idea or topic.

If you embed very large text:

- multiple topics mix together
- semantic meaning becomes blurred

Chunking ensures each embedding represents a focused concept.

Text Splitter

4. Faster Processing
Working with smaller chunks:

- speeds up embedding creation
- speeds up similarity search in vector databases (Chroma, Pinecone, FAISS).

Large documents would make the pipeline slower and expensive.

Text Splitter

We are going to see 3 types of text splitting methods.

1. Character - Based Splitting
2. Token-Based Splitting
3. Semantic / Meaning-Based Splitting

Text Splitter

1. Character - Based Splitting

Character-based splitting is the simplest method of text splitting.
It divides a large document into smaller chunks based on the number of characters, without understanding words, tokens, or meaning.

Text Splitter

2. Token-Based Splitting

Token-based splitting divides a document into chunks based on the number of tokens, not characters.
Since LLMs process tokens instead of words, this method aligns better with how models actually read text.

Text Splitter

3. Semantic / Meaning-Based Splitting

Semantic / Meaning-Based Splitting divides a document into chunks based on the meaning and structure of the text, rather than characters or tokens.
The goal is to keep complete ideas together so that each chunk contains a coherent piece of information.

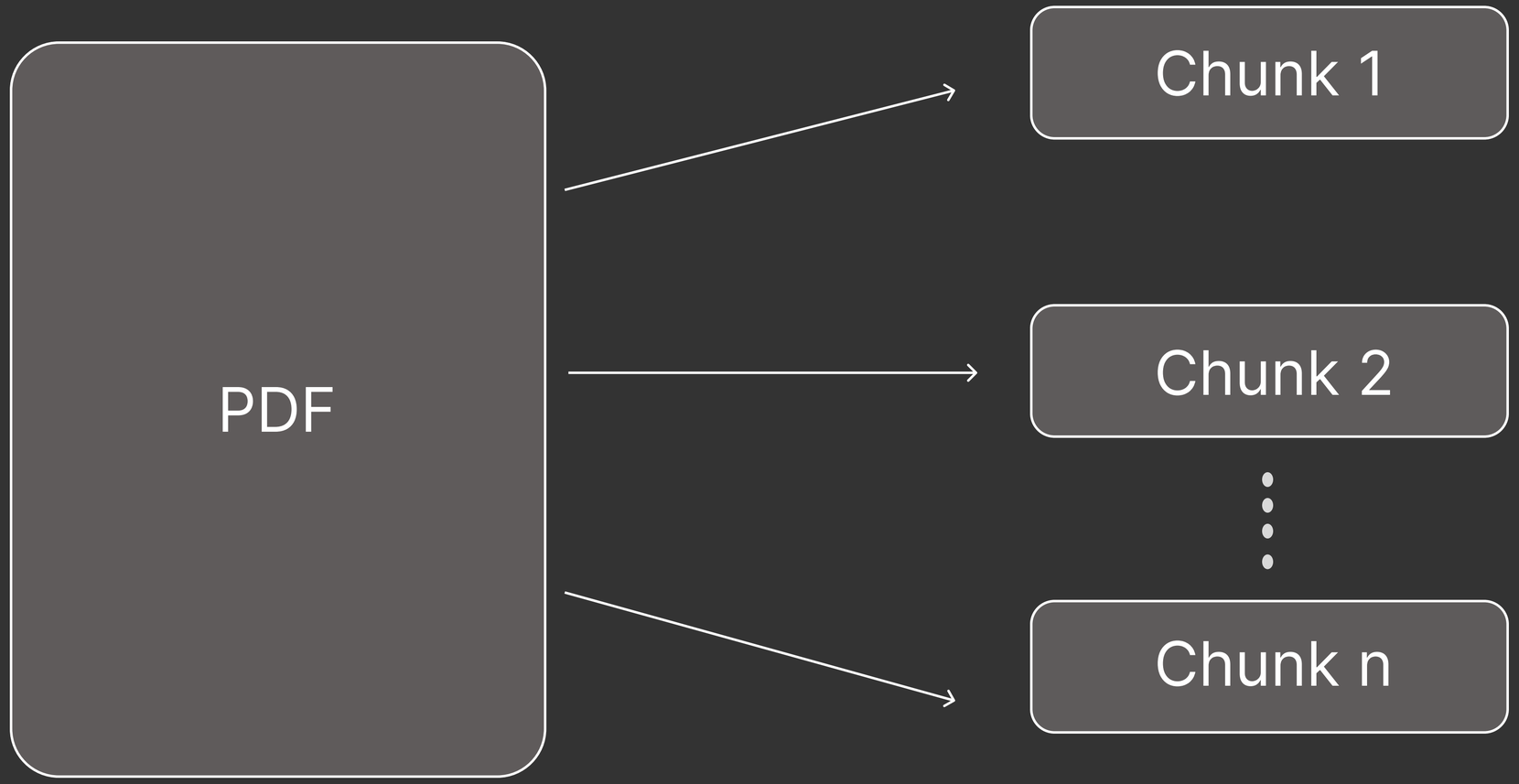
Text Splitter

Deep learning is a specialized area of machine learning that uses neural networks with multiple layers to automatically learn complex representations from large amounts of data. Natural Language Processing (NLP) is another important area of artificial intelligence that focuses on enabling computers to understand, interpret, and generate human language in tasks such as translation, chatbots, and sentiment analysis.

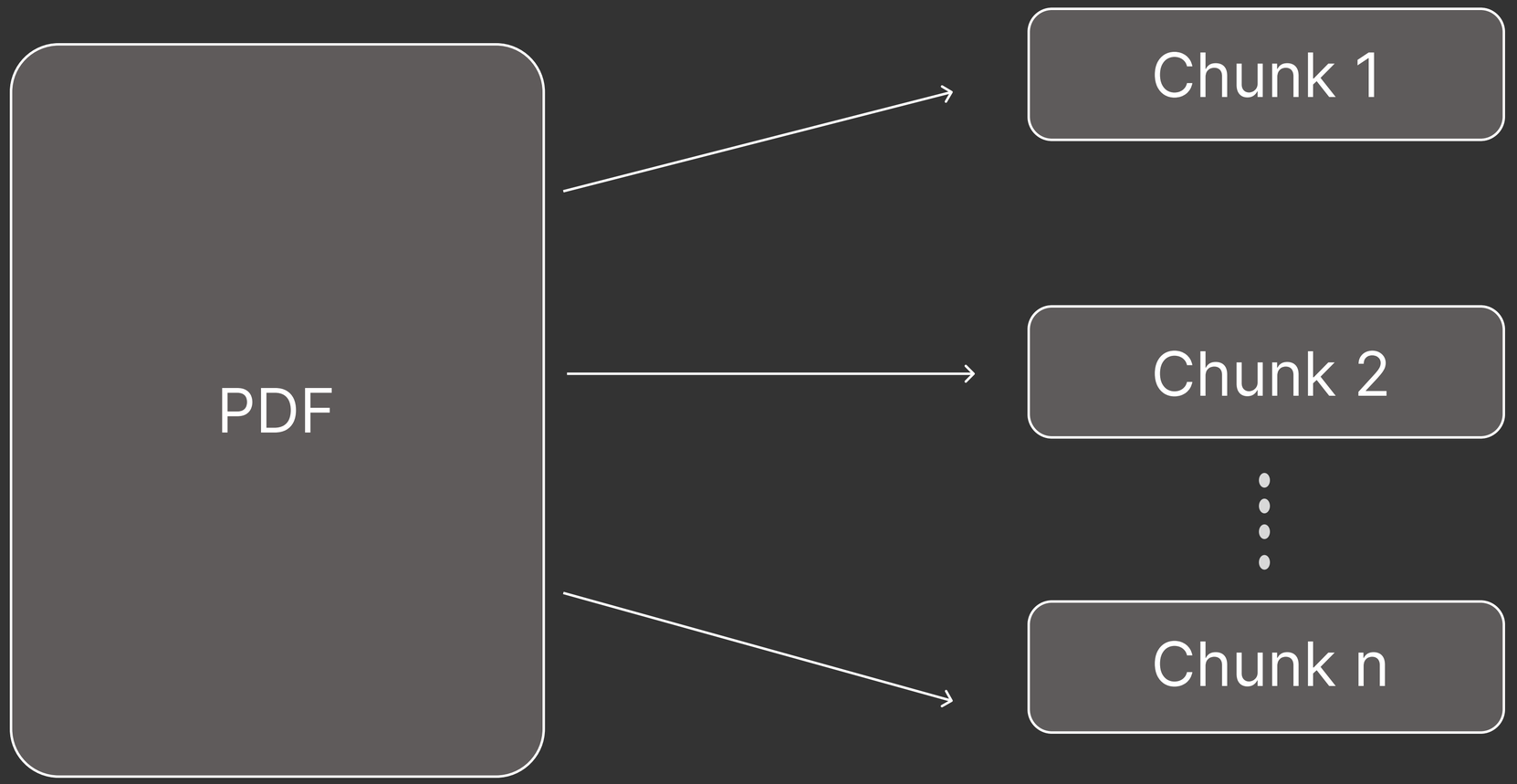
Vector Store

So first of all if we talk about our project we are creating a AI assistant which is going to search our pdf and give answers based on that.

Vector Store

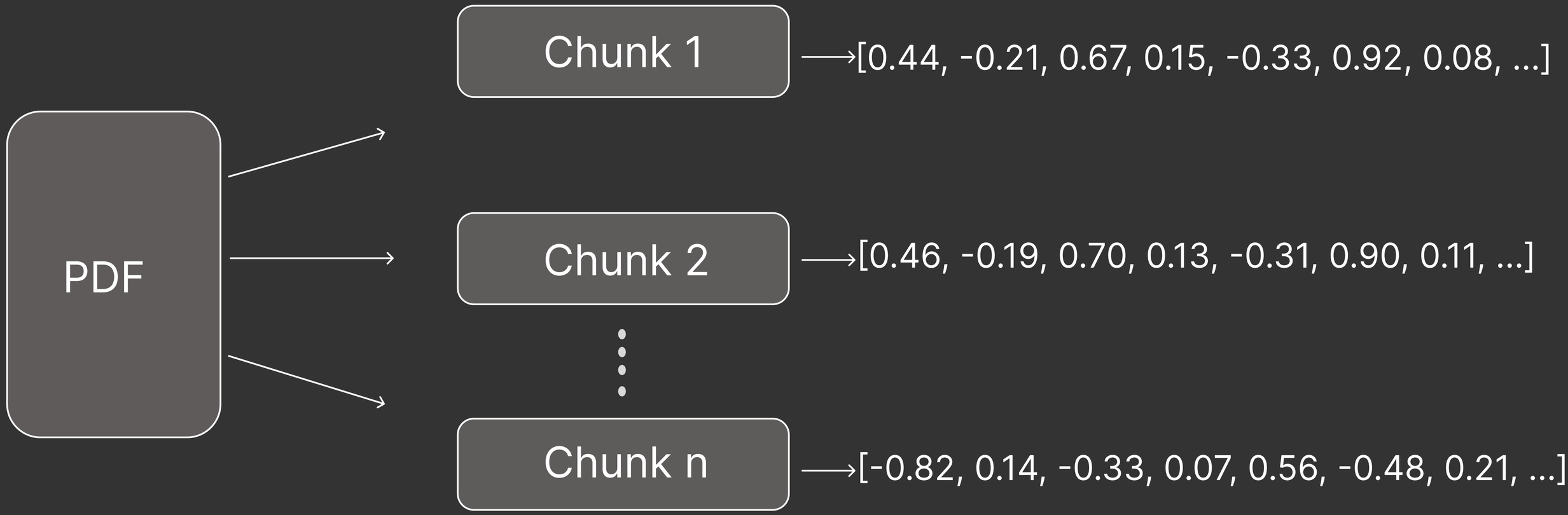


Vector Store

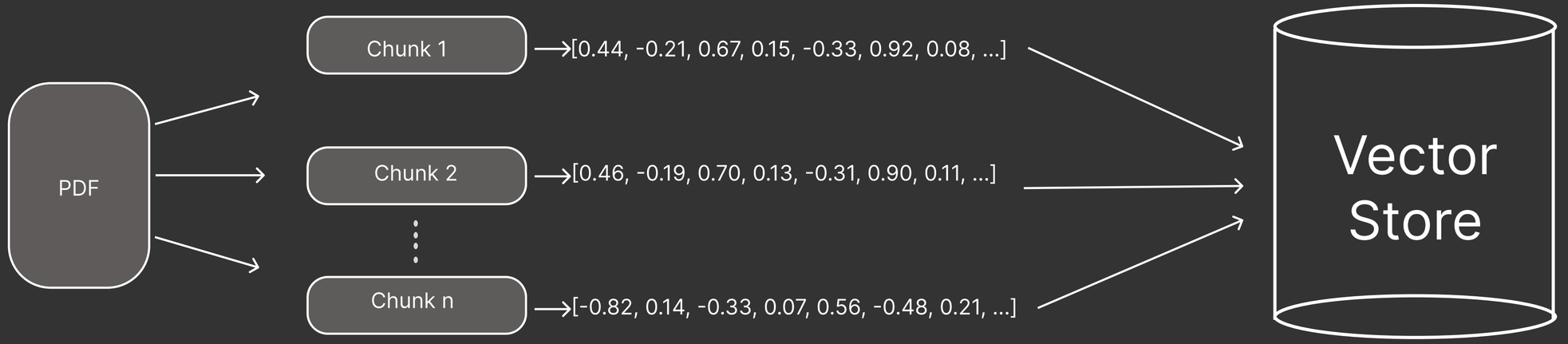


After this our next step will be to create embeddings of these chunks.

Vector Store



Vector Store



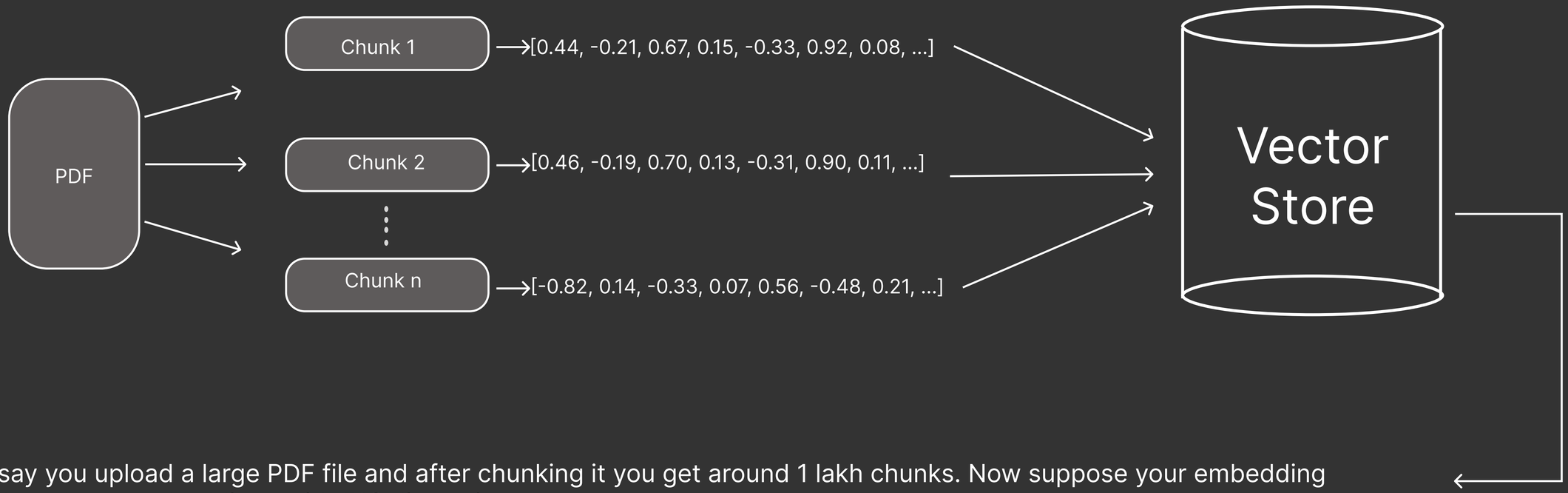
Now after this we need to store these embeddings and for that we are going to use vector store.

Vector Store

Now the question that might come to your mind is that we already have many types of databases like MySQL, PostgreSQL, MongoDB, etc. So why do we even need vector databases?

Vector Store

Answer is actually simple lets understand.



Let's say you upload a large PDF file and after chunking it you get around 1 lakh chunks. Now suppose your embedding model produces embeddings with 512 dimensions.

Vector Store

1 lakh
embedding

Now lets say this is a normal DataBase. and user give a query.

↓

What does the notes say about Natural Language processing.

Vector Store

1 lakh embedding

The Query is first converted into embedding

What does the notes say about Natural Language processing.

↓

[-0.82, 0.14, -0.33, 0.07, 0.56, -0.48, 0.21, ...]

512 - dimensions

Vector Store

1 lakh embedding

The biggest problem is this 512 dimension embedding is different from all the 1 lakh embeddings in our database so we conduct a similarity search with all the 1 lakh embeddings.

[-0.82, 0.14, -0.33, 0.07, 0.56, -0.48, 0.21, ...]

512 - dimensions

Vector Store

1 lakh
embedding

so you are working at 0(n) time complexity and you are searching for 1 lakh time and then finding out and this dataset can become more big so they are not the reliable option for similarity searching.

[-0.82, 0.14, -0.33, 0.07, 0.56, -0.48, 0.21, ...]

512 - dimensions

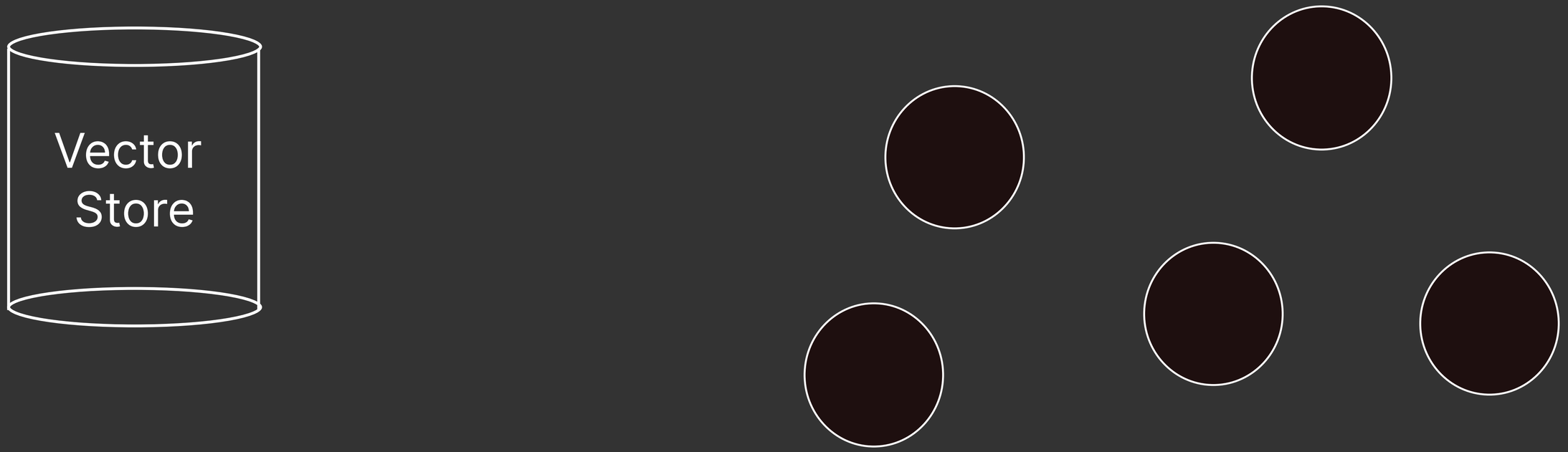
Vector Store

Where as vector store use Approximate Nearest Neighbour algorithms like -
HNSW
IVF
PQ

and all of these are indexing techniques
now we are not going to see all but lets see an example why they are fast.

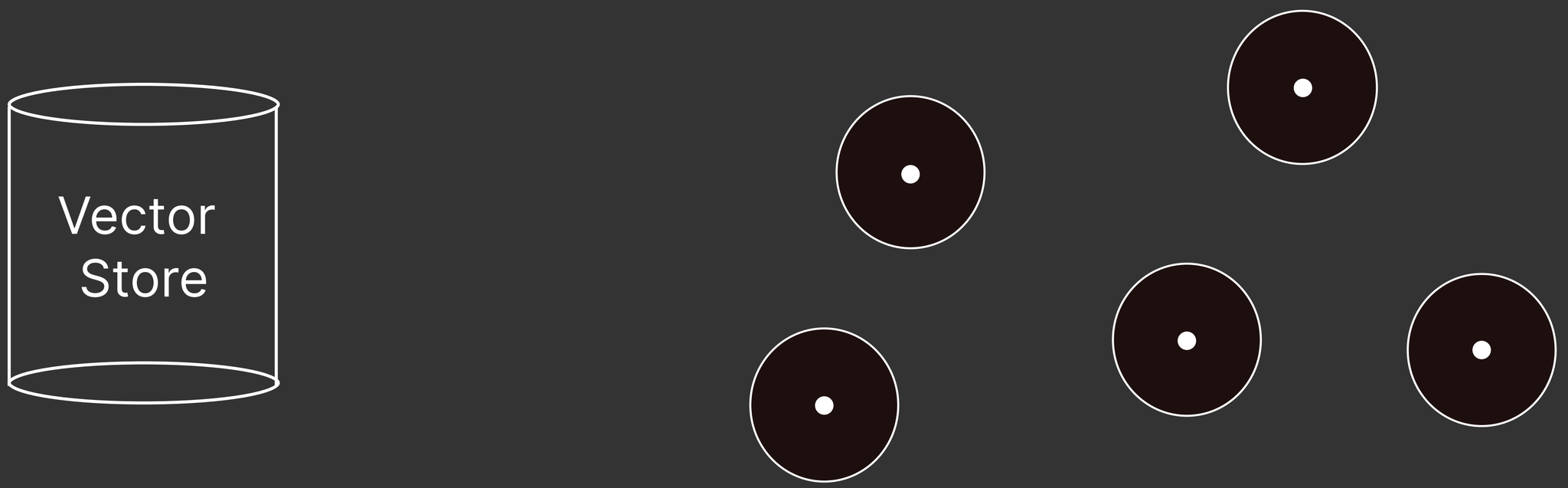
Vector Store

IVF (Inverted File Index) - Lets say we divide our Database in 5 clusters using k-means or any other algo. so each will have 20000 embeddings and all of them have some sort of similarity.



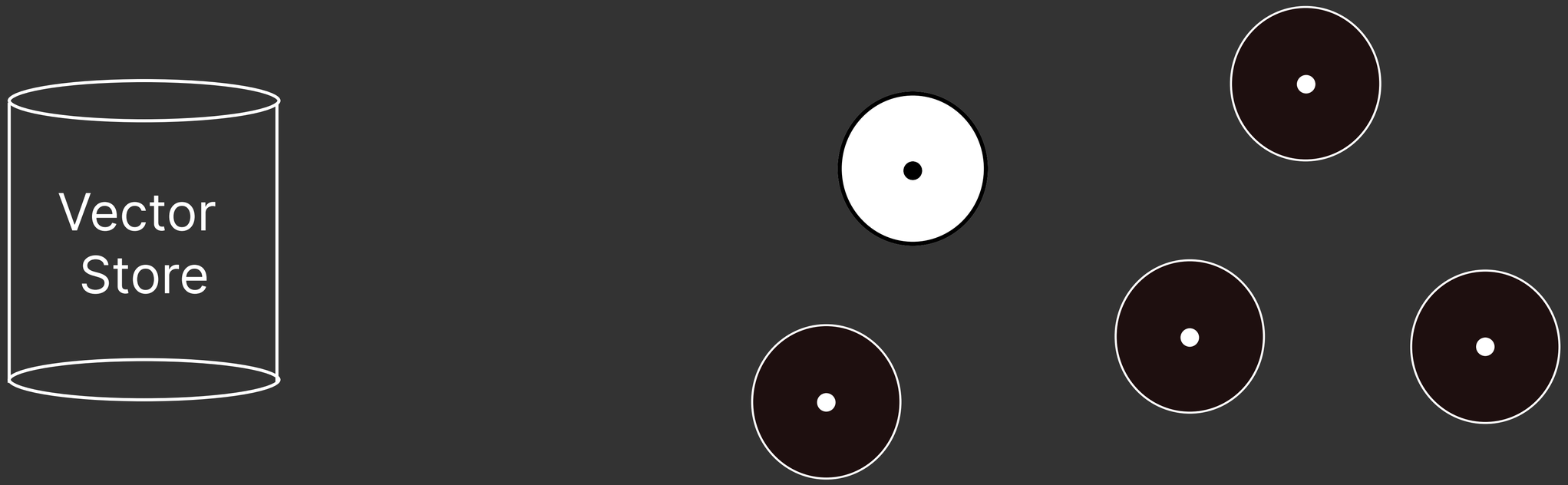
Vector Store

After that we calculate the average embedding of all the 20000 embeddings. we can call it centroid. so now we have 5 average embeddings.



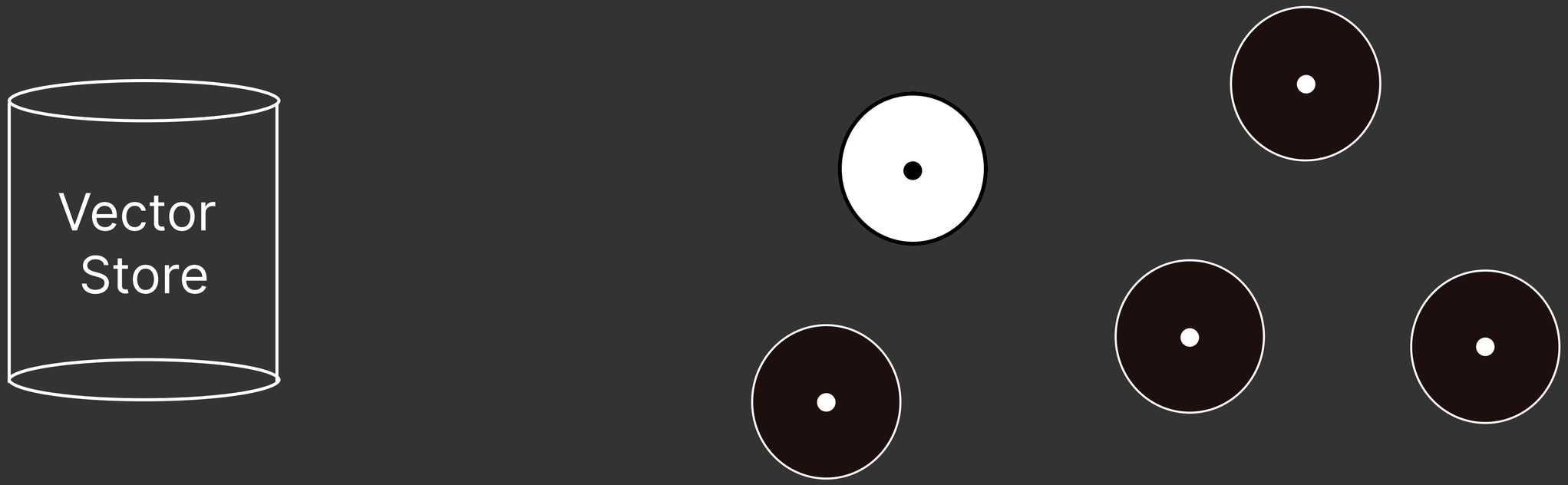
Vector Store

Now for the query embedding we can find which cluster is suitable for searching and we search in those 20000 embeddings. that is actually 5 times faster that normal database



Vector Store

Now for the query embedding we can find which cluster is suitable for searching and we search in those 20000 embeddings. that is actually 5 times faster that normal database



Vector Store

Feature	Normal Database	Vector Database
Data stored	Structured data (text, numbers, rows)	Embeddings (vectors like `[0.23, -0.41, ...]`)
Search type	Exact match	Similarity search
Query example	`WHERE movie = "Interstellar"`	"movies about space travel"
Indexing	B-Tree, Hash	HNSW, IVF, PQ
Use case	Banking, user records, transactions	AI search, recommendations, RAG
Comparison method	Equality or filtering	Cosine similarity / vector distance

Vector Store

Now there are many Vector stores like
These are widely used with frameworks like LangChain and LlamaIndex.

- Chroma
- FAISS
- Annoy

Vector Store

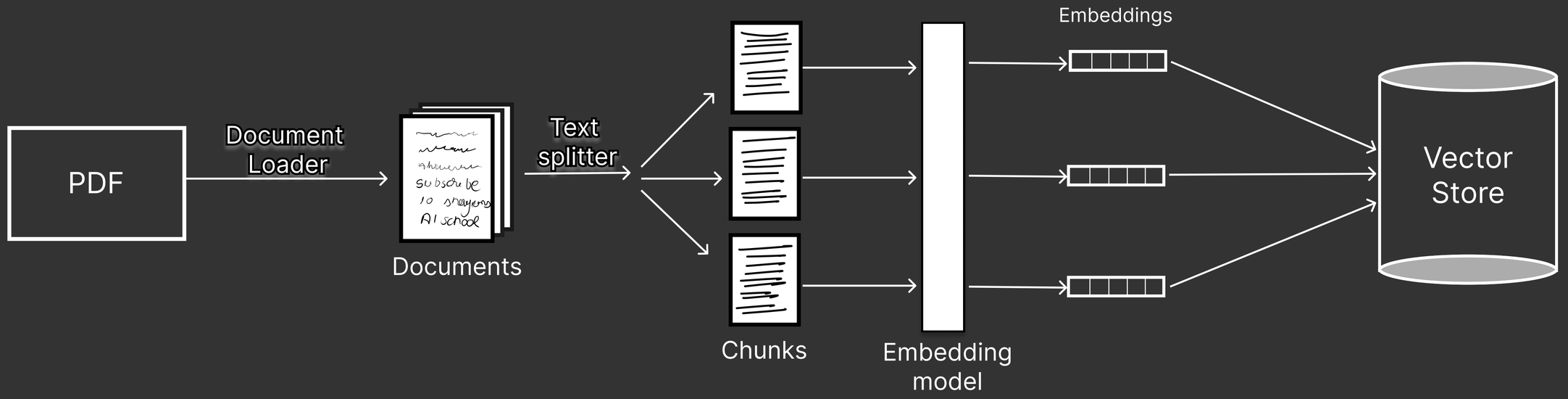
Enough Yapping lets see how to use one Vector Store.

Retrievers

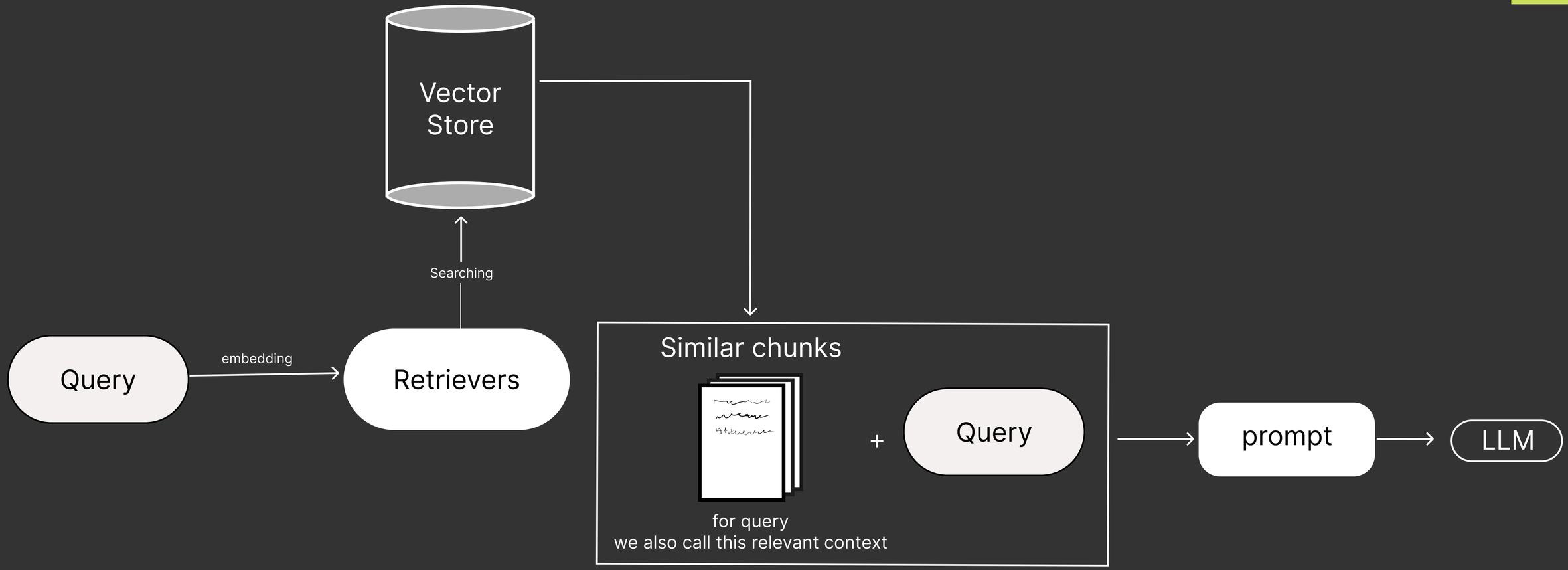
Now before understanding retrievers lets go through a diagram to understand the application that we are creating and we will also see the use of retrievers in that application.

Retrievers

Phase 1



Retrievers


Vector Store
Searching
Query
embedding
Retrievers
Similar chunks
+
Query
for query
we also call this relevant context
prompt
LLM

Retrievers

A retriever takes a user query and returns the most relevant documents or chunks from a database.

Now there are 2 types of retriever

1. by data source (Wikipedia, Arxiv, PubMed, etc.)
2. By retrieval strategy (Similarity, MMR, MultiQuery, etc.)

Retrievers

By data source

When we talk about retrievers using data sources, we mean retrievers that fetch information from specific external sources or databases instead of your own vector store. These retrievers connect to APIs or datasets and return relevant documents for the query.

Lets see an example.

Retrievers

By data source

So I hope now you get the gist how by data source works same code can generate the results from the wikipedia and other types of retrievers by data source.

Retrievers

By Retriever strategy

First go to Vector store code for an important information.

Retrievers

By Retriever strategy

First go to Vector store code for an important information.

Now you know that retrievers are not only used for basic similarity search strategies. They also provide some powerful search strategies that make them more useful, and they are runnables as well, which makes them suitable for building chains.

Retrievers

Search Strategies

In most RAG (Retrieval-Augmented Generation) systems, retrievers mainly use three core retrieval strategies. These are the ones you'll see most often in real projects and tutorials.

1. Similarity Search (Most Common)
2. MMR (Max Marginal Relevance)
3. MultiQuery Retriever

Retrievers

Search Strategies - Similarity Search

The system compares the query vector with document vectors using similarity metrics like:

- Cosine similarity (most common)
- Dot product
- Euclidean distance

The retriever finds the most similar vectors.

Top-K documents are retrieved

Retrievers

Search Strategies - MMR (maximum marginal relevance)

Lets understand this with an example :
Imagine you are building a RAG system for your students.
Your knowledge base contains many chunks about Gradient Descent.
Your database chunks look like this:

Chunk 1: Gradient descent is an optimization algorithm used in machine learning.
Chunk 2: Gradient descent minimizes the loss function.
Chunk 3: Gradient descent is an optimization that minimizes the loss function.
Chunk 4: Neural networks use gradient descent for training.
Chunk 5: Support Vector Machines are supervised learning algorithms.

Retrievers

Now the user asks
What is Gradient Descent ?

Normal Similarity search finds the most similar chunks to the query embedding.
and gives chunk 1 , chunk 2 , chunk 3

but if you see carefully all the 3 chunks are saying the same thing.

This wastes:

- context window
- token usage
- information diversity

The LLM doesn't learn new information.

Retrievers

MMR stands for:
Max Marginal Relevance

Its goal is to balance two things:
1. Relevance to the query
2. Diversity among retrieved documents

So instead of retrieving similar documents, it retrieves different but relevant documents.

Retrievers

Search Strategies - MultiQuery

Imagine you built a RAG system for machine learning notes.

Your database contains these chunks:

Chunk 1: Gradient descent is an optimization algorithm used to minimize loss functions.
Chunk 2: Neural networks are trained using optimization algorithms like gradient descent.
Chunk 3: Weight updates in machine learning models are done using gradient descent.
Chunk 4: Support Vector Machines are supervised learning algorithms.

Retrievers

Chunk 1: Gradient descent is an optimization algorithm used to minimize loss functions.
Chunk 2: Neural networks are trained using optimization algorithms like gradient descent.
Chunk 3: Weight updates in machine learning models are done using gradient descent.
Chunk 4: Support Vector Machines are supervised learning algorithms.

Now the user asks:
"What is gradient descent?"

the retriever might return:
Chunk 1
Chunk 3

Retrievers

Chunk 1: Gradient descent is an optimization algorithm used to minimize loss functions.
Chunk 2: Neural networks are trained using optimization algorithms like gradient descent.
Chunk 3: Weight updates in machine learning models are done using gradient descent.
Chunk 4: Support Vector Machines are supervised learning algorithms.

But notice something
There are other relevant chunks like Chunk 2 that might not be retrieved.
Why?
Because the query wording is limited.

Retrievers

The Core Problem
A single query might not capture all ways a concept can be expressed.
for example :
"What is gradient descent?"

could be asked as

"Explain gradient descent"
"How does gradient descent work?"
"Optimization algorithm used in neural networks"

Retrievers

The Core Problem
A single query might not capture all ways a concept can be expressed.
for example :
"What is gradient descent?"

could be asked as

"Explain gradient descent"
"How does gradient descent work?"
"Optimization algorithm used in neural networks"

Different wording → different embeddings → different search results.
This is where MultiQuery Retriever helps.

Retrievers

So Instead of using one query, it generates multiple variations of the query using an LLM. so now the pipeline becomes

User Query
↓
LLM generates multiple queries
↓
Retriever searches for each query
↓
Combine all retrieved documents

Retrievers

So now lets finally complete our project.